

Introduction to Machine Learning

Lecture 16: Generalization

Ali Bereyhi

`ali.bereyhi@utoronto.ca`

Department of Electrical and Computer Engineering
University of Toronto

Winter 2026

Recap: *Building and Training NN*

TrainingLoop():

- 1: Build NN $y = f_w(x)$ with some **hyperparameters** and *initial weights*
- 2: Split training set to mini-batches
- 3: Specify the loss function $\mathcal{L}$
- 4: **for** $epochs = 1, \dots, E$ **do**
- 5: Keep applying mini-batch SGD
- 6: **end for**
- 7: Return final weights w^* , average training loss, and accuracy on training set

Recap: Validation

Validation():

- 1: Realize NN $\mathbf{y} = f_{\mathbf{w}^*}(\mathbf{x})$ with *trained weights* $\mathbf{w}^*$
- 2: **for** sample i in validation set **do**
- 3: Compute $\mathbf{y}_i = f_{\mathbf{w}^*}(\mathbf{x}_i)$
- 4: Compute sample loss $\hat{R}_i = \mathcal{L}(\mathbf{y}_i, \mathbf{v}_i)$
- 5: Check whether inferred label $\hat{v}_i$ is the same as true label $\mathbf{v}_i$
- 6: **end for**
- 7: Return average validation loss and accuracy

We repeat training with *new hyperparameters* till it's tuned

Recap: *Evaluation*

Evaluation():

- 1: Realize NN $\mathbf{y} = f_{\mathbf{w}^*}(\mathbf{x})$ with *trained weights $\mathbf{w}^*$* and *tuned hyperparameters*
- 2: **for** sample i in evaluation set **do**
- 3: Compute $\mathbf{y}_i = f_{\mathbf{w}^*}(\mathbf{x}_i)$
- 4: Compute sample loss $\hat{R}_i = \mathcal{L}(\mathbf{y}_i, \mathbf{v}_i)$
- 5: Check whether inferred label $\hat{v}_i$ is the same as true label $\mathbf{v}_i$
- 6: **end for**
- 7: Return average evaluation loss and accuracy

Today's Agenda: *Generalization and Convolution*

Today, we talk about

Generalization

In this way, we discuss the following topics

- *Generalization concept and bias-variance trade-off*
- *Memory-capacity trade-off and overfitting*
- *Techniques to reduce overfitting*

Trained Model on New Data

Once we are finished with training, we use our model for inference

we hope our model works fine on unseen samples as well

Generalization

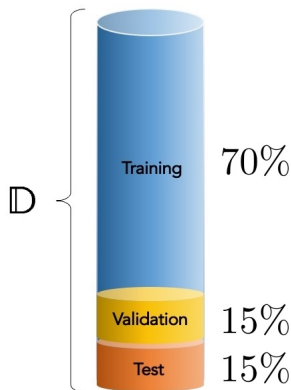
*Ability of a trained model to perform well on **unseen** data*

Unseen data is coming from a randomly and its label is unknown to us!

Measuring Generalization

? *How can we measure generalization?*

! *We try samples that we did not use for training*



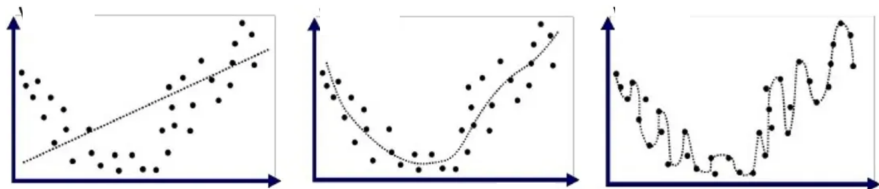
Underfitting and Overfitting

If model does not generalize we see one of these cases

- *Training loss is small but validation loss is large! $\rightsquigarrow$ Overfitting*
 - ↳ *Model fits dataset $\rightsquigarrow$ it memorizes dataset*
 - ↳ *It does not generalize to new data $\rightsquigarrow$ it did not learn properly*
- *Training loss is large and validation loss is large! $\rightsquigarrow$ Underfitting*
 - ↳ *Model does not fit dataset $\rightsquigarrow$ it cannot memorize dataset*
 - ↳ *It does not generalize to new data $\rightsquigarrow$ it did not learn properly*

Nowadays, we mainly observe overfitting!

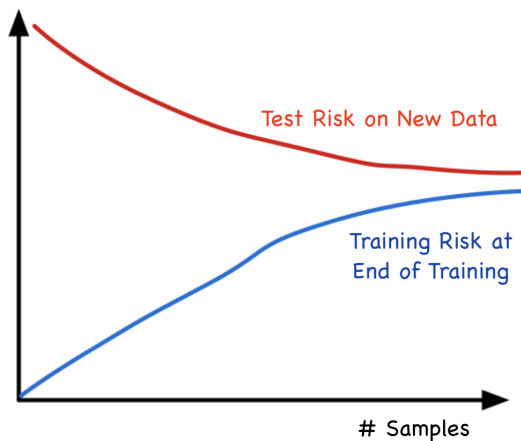
Underfitting and Overfitting



- Left $\rightsquigarrow$ Underfitting
- Middle $\rightsquigarrow$ Learning fairly
- Right $\rightsquigarrow$ Overfitting

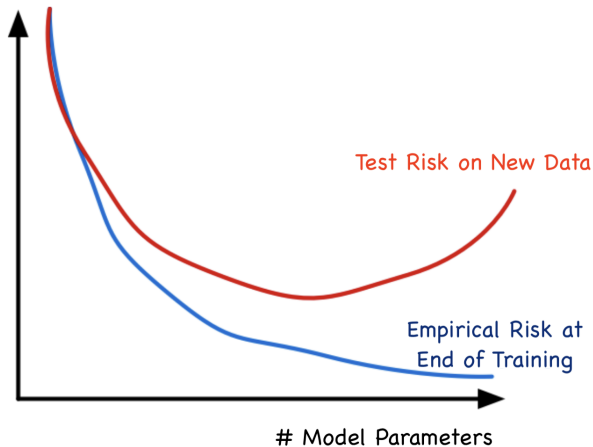
Generalization vs Data

Adding more data reduces chance of memorizing and increase learning ability



Generalization vs Model Capacity

More model parameters increase both memorizing and learning abilities



Formulating Generalization

Let us consider a simple case: *we train a model $f_{\mathbf{w}}(\cdot)$ on dataset*

$$\mathbb{D} = \{(\mathbf{x}_n, v_n) : n \in [N]\}$$

We evaluate generalization by squared error: for a new point $\mathbf{x}$

$$\mathcal{E} = \mathbb{E} \left\{ (y - v)^2 \right\}$$

- *y is the prediction given by our trained model*

$$y = f_{\mathbf{w}}(\mathbf{x})$$

- *v is the label that we do not know*

Generalization Error

Assume that samples are drawn i.i.d. from a data distribution: *each sample*

$$(\mathbf{x}, v) \sim P(\mathbf{x}, v)$$

We can use Bayes' rule to write

$$P(\mathbf{x}, v) = P(\mathbf{x}) P(v|\mathbf{x})$$

- $P(\mathbf{x})$ is the distribution of data-points
- $P(v|\mathbf{x})$ is the distribution of labels when $\mathbf{x}$ is known

Attention

We do not have access to $P(\mathbf{x})$ or $P(v|\mathbf{x})$ in practice! They are hypothetical

Generalization Error: *Single Sample*

Assume that we only care about sample $(\mathbf{x}, v)$: we know $\mathbf{x}$ but not v

$$\begin{aligned}\mathcal{E}(\mathbf{x}) &= \mathbb{E} \left\{ (y - v)^2 | \mathbf{x} \right\} \\ &= \mathbb{E} \left\{ y^2 - 2yv + v^2 | \mathbf{x} \right\} \\ &= \mathbb{E} \left\{ y^2 | \mathbf{x} \right\} - 2\mathbb{E} \left\{ yv | \mathbf{x} \right\} + \mathbb{E} \left\{ v^2 | \mathbf{x} \right\}\end{aligned}$$

Since we know $\mathbf{x}$, we know $y = f_{\mathbf{w}}(\mathbf{x})$ as well

$$\mathcal{E}(\mathbf{x}) = y^2 - 2y\mathbb{E} \{ v | \mathbf{x} \} + \mathbb{E} \{ v^2 | \mathbf{x} \}$$

Generalization Error: *Single Sample*

The error for a single sample is

$$\mathcal{E}(\mathbf{x}) = y^2 - 2y\mathbb{E}\{v|\mathbf{x}\} + \mathbb{E}\{v^2|\mathbf{x}\}$$

Let's define the mean and variance of v determined by $P(v|\mathbf{x})$ as

$$\mu(\mathbf{x}) = \mathbb{E}\{v|\mathbf{x}\} \quad \sigma^2(\mathbf{x}) = \mathbb{E}\{v^2|\mathbf{x}\} - \mu^2(\mathbf{x})$$

Then, we can write

$$\begin{aligned}\mathcal{E}(\mathbf{x}) &= y^2 - 2y\mu(\mathbf{x}) + \mu^2(\mathbf{x}) + \sigma^2(\mathbf{x}) \\ &= [y - \mu(\mathbf{x})]^2 + \sigma^2(\mathbf{x}) \\ &= \text{Model Bias}^2 + \text{Label Variance}\end{aligned}$$

Generalization Error: *Bias and Variance*

Generalization Error

Generalization error for single new sample x is

$$\mathcal{E}(x) = [y - \mu(x)]^2 + \sigma^2(x)$$

The only thing that we can control is y

- *We can hope that our model gives us zero bias $\rightsquigarrow$ unbiased estimator*

$$y = f_{\mathbf{w}}(x) = \mathbb{E}\{v|x\} = \mu(x)$$

- *We can do nothing about $\sigma^2(x)$ $\rightsquigarrow$ Bayes error*

Bayes Error

Minimum theoretically possible generalization error for a learning problem

Generalization Error: *Expected*

But, we don't care about a **single** point: we *care about expected error*

$$\mathbb{E} \{ \mathcal{E}(\mathbf{x}) \} = \mathbb{E} \left\{ [y - \mu(\mathbf{x})]^2 \right\} + \mathbb{E} \{ \sigma^2(\mathbf{x}) \}$$

Let's define the mean and variance of the model output

$$\bar{y} = \mathbb{E} \{ y \} \quad \text{Var} \{ y \} = \mathbb{E} \left\{ (y - \bar{y})^2 \right\} = \mathbb{E} \{ y^2 \} - \bar{y}^2$$

We can then write

$$\begin{aligned} \mathbb{E} \{ \mathcal{E}(\mathbf{x}) \} &= \mathbb{E} \left\{ [y - \bar{y} + \bar{y} - \mu(\mathbf{x})]^2 \right\} + \mathbb{E} \{ \sigma^2(\mathbf{x}) \} \\ &= \mathbb{E} \left\{ (y - \bar{y})^2 \right\} + 2\mathbb{E} \{ (y - \bar{y})(\bar{y} - \mu(\mathbf{x})) \} + \mathbb{E} \left\{ (\bar{y} - \mu(\mathbf{x}))^2 \right\} \\ &\quad + \mathbb{E} \{ \sigma^2(\mathbf{x}) \} \end{aligned}$$

Generalization Error: *Expected*

Let's further define the mean and variance of the label

$$\bar{v} = \mathbb{E} \{v\} = \mathbb{E} \{\mu(\mathbf{x})\} \quad \text{Var} \{v\} = \mathbb{E} \{(v - \bar{v})^2\} = \mathbb{E} \{\sigma^2(\mathbf{x})\}$$

We can then repeat the same decomposition and finally write approximately

$$\begin{aligned} \mathbb{E} \{\mathcal{E}(\mathbf{x})\} &\approx (\bar{y} - \bar{v})^2 + \text{Var} \{y\} + \text{Var} \{v\} \\ &= \text{Model Bias}^2 + \text{Model Variance} + \text{Label Variance} \end{aligned}$$

Generalization Error Components

Generalization error is proportional to the model bias and variance

Generalization Error: *Bias and Variance*

The only thing that we can control is y

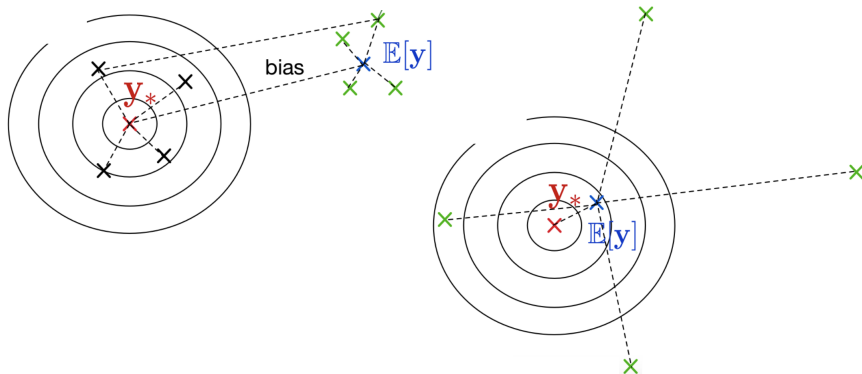
- *We set the model to give us zero bias $\rightsquigarrow$ unbiased estimator*

$$\bar{y} = \mathbb{E} \{f_{\mathbf{w}}(\mathbf{x})\} = \mathbb{E} \{v\} = \bar{v}$$

- *But this may lead to higher variance $\text{Var} \{y\}$*
- *There is always a minimum error that we cannot beat $\rightsquigarrow$ Bayes error*

Bias and Variance: Visualization

We can visualize it as¹



¹Borrowed from ML lecture-notes by R. Grosse

Reducing Overfitting

Most basic approach is to tune hyperparameters

Hyperparameter

Any parameter we set to realize our model, e.g., depth, width, etc

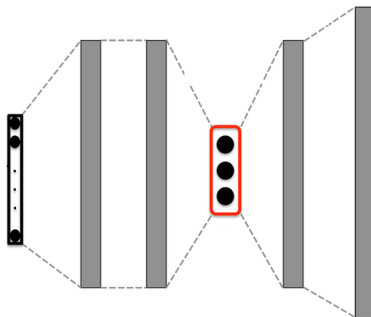
Start with initial setting

- *Train the model and save the training loss*
- *Evaluate it on the validation dataset*
 - ↳ *If the loss is too different from training loss*
 - ↳ *change hyperparameters and re-train*
 - ↳ *Else*
 - ↳ *training is over*

? *How can we search?*

! *Depending on the complexity: random or grid search*

Bottleneck



Bottleneck can drop the model capacity

- Say we add a 10-neuron layer between two 200-neuron layers
 - ↳ Without bottleneck we have 40,000 weights
 - ↳ With bottleneck we have 4,000 weights!

Regularization: *General Form*

In regularization, we add a penalty to empirical risk: *training minimizes*

$$\tilde{R}(\mathbf{w}) = \frac{1}{N} \sum_{n=1}^N \mathcal{L}(y_n, v_n) + \lambda \Pi(\mathbf{w})$$

Penalty is proportional to the property of weights when model overfits

Example

If we know that weights get large when model overfits we could set

$$\Pi(\mathbf{w}) = \|\mathbf{w}\|^2 = \sum_i w_i^2$$

- We typically pre-set the regularization function, i.e., penalty
- λ is a hyperparameter that we call *regularizer* and tune by validation

Regularization: ℓ_p -Norm

Famous choice of penalty is ℓ_p -norm: we set

$$\Pi(\mathbf{w}) = \|\mathbf{w}\|_p^p = \sum_i |w_i|^p$$

- ℓ_2 -norm is the classic choice $\rightsquigarrow$ Tikhonov
 - ↳ It avoids weights to get very large
 - ↳ You did it in Assignment 2 for regression $\rightsquigarrow$ Ridge regression
- ℓ_1 -norm is another classic choice $\rightsquigarrow$ Lasso
 - ↳ It enforces the weights to be sparse $\rightsquigarrow$ contain too many zeros
 - ↳ Great piece of work by Robert Tibshirani 😊

Regularization and Weight Decay

Let's set $\Pi(\mathbf{w}) = \|\mathbf{w}\|^2/2$ and write GD

$$\begin{aligned}\mathbf{w} &\leftarrow \mathbf{w} - \eta \nabla \tilde{R}(\mathbf{w}) \\ &\leftarrow \mathbf{w} - \eta \nabla \left(\frac{1}{N} \sum_{n=1}^N \mathcal{L}(y_n, v_n) + \frac{\lambda}{2} \|\mathbf{w}\|^2 \right)\end{aligned}$$

Now focus on a single w_i : it would be like a **weight decay**

$$\begin{aligned}w_i &\leftarrow w_i - \eta \left(\frac{\partial}{\partial w_i} \hat{R}(\mathbf{w}) + \frac{\lambda}{2} \frac{\partial}{\partial w_i} \|\mathbf{w}\|^2 \right) \\ &\leftarrow w_i - \eta \lambda w_i - \eta \frac{\partial}{\partial w_i} \hat{R}(\mathbf{w}) = w_i (1 - \eta \lambda) - \eta \frac{\partial}{\partial w_i} \hat{R}(\mathbf{w})\end{aligned}$$

Some people call ℓ_2 -norm regularization weight decay

Stochastic Regularization

We can also apply stochastic regularization: *stating training with SGD*

- In each gradient computation
 - ↳ Apply a stochastic operation on neuron outputs
- Average all sample gradients over mini-batch
- Apply an iteration of GD

The most famous approach is *dropout*

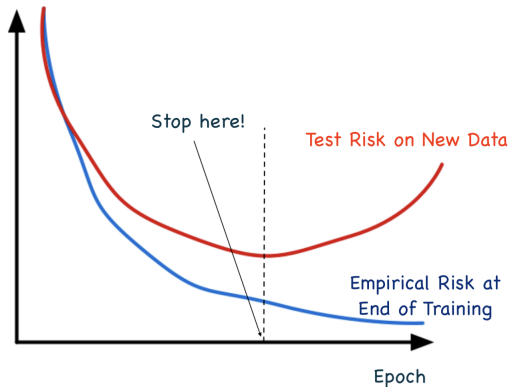
- In each gradient computation
 - ↳ we randomly drop some neurons

$$y_i \leftarrow m_i y_i$$

where $m_i \in \{0, 1\}$ is a random mask

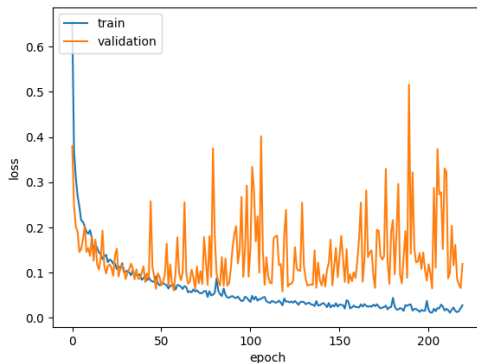
Early Stopping

We may stop when we the model is well-trained: *validate after each epoch*



Early Stopping

In practice though it's hard: *we usually see*



We may use moving average; however, it's not always effective

Ensemble

We could use multiple model: *give the task to m model*

- *Each model predicts a label y_i*
- *We set the average as the final prediction*

$$y = \frac{1}{m} \sum_{i=1}^m y_i$$

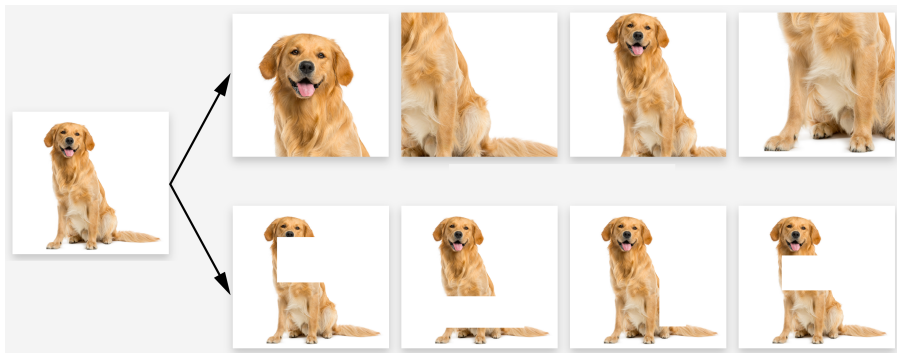
? *How to realize multiple models?*

We can take different approaches

- *Bagging $\rightsquigarrow$ use the same model on different data*
- *Stacking $\rightsquigarrow$ use different models on the same data*
- *Boosting $\rightsquigarrow$ modify the data based on previous predictions and re-train*

Data Augmentation

We can augment to increase samples



Further Read

- Goodfellow
 - ↳ Chapter 6

Regularization